

Ví dụ bài tập liệt kê:

So sánh Phương pháp sinh và Quay lui

1 Phương pháp sinh trong bài toán liệt kê

Phương pháp sinh là gì?

Ý tưởng chung của phương pháp sinh là: tìm cách **sinh trực tiếp** các nghiệm theo một thứ tự nào đó, từ nghiệm đầu tiên đến nghiệm cuối cùng.

Nói cách khác, ta tổ chức tập nghiệm thành một dãy:

$$x^{(1)}, x^{(2)}, x^{(3)}, \dots$$

rồi tìm quy tắc để đi từ nghiệm hiện tại sang **ngiem kế tiếp**.

Bốn câu hỏi quan trọng

Khi giải một bài toán bằng phương pháp sinh, ta thường phải trả lời ba câu hỏi:

1. **Biểu diễn nghiệm như thế nào?** Mỗi nghiệm sẽ được viết dưới dạng gì: một xâu nhị phân, một tổ hợp tăng dần, một hoán vị,...
2. **Nghiem đầu tiên là gì?** Ta sẽ bắt đầu quá trình sinh từ cấu hình nào.
3. **Sinh nghiệm kế tiếp như thế nào?** Từ một nghiệm hiện tại, cần có quy tắc rõ ràng để tạo ra nghiệm tiếp theo.
4. **Khi nào thuật toán dừng?** Cần xác định nghiệm cuối cùng, hoặc ít nhất là điều kiện nhận biết rằng không còn nghiệm kế tiếp.

Khi nào nên nghĩ đến phương pháp sinh?

Phương pháp sinh đặc biệt phù hợp khi:

- tập nghiệm có **cấu trúc rõ ràng**;
- các nghiệm có thể **sắp theo thứ tự**;
- tồn tại một quy tắc đơn giản để chuyển từ nghiệm này sang nghiệm kế tiếp.

Khuôn mẫu tổng quát

Một thuật toán sinh thường có dạng:

Algorithm 1 Khuôn mẫu tổng quát của phương pháp sinh

```
1: Khởi tạo nghiệm đầu tiên  $x$ 
2:  $done \leftarrow \text{false}$ 
3: while not  $done$  do
4:   In nghiệm hiện tại  $x$ 
5:   if  $x$  là nghiệm cuối cùng then
6:      $done \leftarrow \text{true}$ 
7:   else
8:     Sinh nghiệm kế tiếp từ  $x$ 
9:   end if
10: end while
```

Nhận xét

Ưu điểm của phương pháp sinh là làm nổi bật **cấu trúc tổ hợp** của bài toán. Tuy nhiên, không phải bài toán nào cũng dễ tìm được quy tắc sinh nghiệm kế tiếp. Trong những trường hợp ràng buộc phức tạp, quay lui thường là cách tiếp cận tự nhiên hơn.

2 Phương pháp quay lui trong bài toán liệt kê

Trong các bài toán liệt kê, **quay lui** là một phương pháp rất tự nhiên và thường được dùng khi nghiệm có thể được **xây dựng từng bước**.

Phương pháp quay lui là gì?

Ý tưởng chung của phương pháp quay lui là: xây dựng nghiệm dần dần từng thành phần một; tại mỗi bước, thử các khả năng có thể xảy ra, rồi tiếp tục mở rộng phần nghiệm đã có.

Nếu ở một thời điểm nào đó, ta nhận thấy phần nghiệm hiện tại **không thể dẫn đến nghiệm đúng**, thì ta dừng nhánh đó và quay lại thử lựa chọn khác. Chính vì vậy, phương pháp này được gọi là **quay lui**.

Ba câu hỏi quan trọng

Khi giải một bài toán bằng phương pháp quay lui, ta thường phải trả lời ba câu hỏi:

1. **Nghiệm được xây dựng theo từng bước như thế nào?** Ta sẽ thêm từng phần tử, từng ký tự, hay từng quyết định nào vào nghiệm.
2. **Ở mỗi bước có những lựa chọn nào?** Tức là tại vị trí hiện tại, ta có thể thử những giá trị nào.
3. **Khi nào cần cắt nhánh?** Nếu phần nghiệm đang có chắc chắn không thể tạo thành nghiệm đúng, ta nên dừng ngay để tránh xét thừa.

Khuôn mẫu tổng quát

Một thuật toán quay lui thường có dạng:

Algorithm 2 Khuôn mẫu tổng quát của phương pháp quay lui

```
1: function TRY(trạng thái hiện tại)
2:   if đã xây dựng xong một nghiệm hoàn chỉnh then
3:     In nghiệm
4:     return
5:   end if
6:   for mỗi lựa chọn có thể tại bước hiện tại do
7:     if lựa chọn này hợp lệ then
8:       Ghi nhận lựa chọn vào nghiệm đang xây dựng
9:       TRY(trạng thái mới)
10:      Quay lại để thử lựa chọn khác
11:    end if
12:  end for
13: end function
```

Thuật toán sẽ đi sâu dần theo một nhánh, đến khi không thể đi tiếp thì quay lại nút trước đó để thử nhánh khác.

Khi nào nên nghĩ đến quay lui?

Phương pháp quay lui đặc biệt phù hợp khi:

- nghiệm được tạo ra **từng bước**;
- tại mỗi bước có **nhiều lựa chọn**;
- bài toán có các **điều kiện ràng buộc** cho phép loại sớm những nhánh không hợp lệ.

3 So sánh 2 phương pháp

- **Quay lui**: xây dựng nghiệm từng phần; tại mỗi bước thử các khả năng rồi quay lại nếu cần.
- **Phương pháp sinh**: xem mỗi nghiệm như một cấu hình hoàn chỉnh; từ nghiệm hiện tại sinh trực tiếp ra nghiệm kế tiếp.

4 Bài toán 1

Cho hai số tự nhiên n và k ($1 \leq k \leq n$). Hãy **liệt kê tất cả các xâu nhị phân độ dài n sao cho mỗi xâu có duy nhất một dãy k bit 1 liên tiếp** (hiểu là **một khối 1 cực đại** có độ dài đúng k).

Ví dụ

Với $n = 7, k = 2$, một nghiệm là 1011001 (các khối 1 có độ dài 1, 2, 1 nên chỉ có đúng một khối dài 2).

Một xâu không hợp lệ là 110011 (có hai khối 11), hoặc 1110000 (khối 1 dài 3 > 2).

Cách 1: Quay lui (Backtracking)

Ý tưởng

Ta xây dựng xâu nhị phân từ trái sang phải. Tại mỗi vị trí, thử đặt 0 hoặc 1, nhưng cắt tỉa sớm những trường hợp chắc chắn sai.

Để làm được điều đó, ta theo dõi hai trạng thái:

- *len*: độ dài khối 1 liên tiếp hiện tại ở cuối tiền tố đang xây dựng;
- *found*: đã **kết thúc** một khối 1 cực đại có độ dài đúng k hay chưa.

Ở đây cần lưu ý rằng một khối 1 dài đúng k chỉ được xem là đã **kết thúc** khi:

- ta đặt một số 0 ngay sau nó, hoặc
- xâu kết thúc.

Cắt tỉa

Trong quá trình quay lui, ta loại ngay các nhánh không hợp lệ:

- không cho xuất hiện khối 1 dài hơn k ;
- không cho xuất hiện khối 1 dài đúng k lần thứ hai.

Giải thích trạng thái

Giả sử đang gọi

$$\text{Try}(i, \text{len}, \text{found}),$$

thì điều đó có nghĩa là:

- các vị trí $x_1, x_2, \dots, x_{i-1}$ đã được xác định;
- *len* là số lượng chữ 1 liên tiếp ở cuối đoạn $x_1 x_2 \dots x_{i-1}$;
- *found* = **true** nếu trước đó đã có đúng một khối 1 cực đại dài k được khép lại.

Giải thích từng nhánh

- **Khi đặt 0**: nếu trước đó $\text{len} = k$, thì khối 1 hiện tại vừa được khép lại và là một khối cực đại dài đúng k , nên phải cập nhật $\text{found} \leftarrow \text{true}$. Sau khi đặt 0, khối 1 ở cuối bị ngắt, do đó trạng thái mới là $\text{len} = 0$.
- **Khi đặt 1**: chỉ được phép nếu $\text{len} + 1 \leq k$, nhằm tránh tạo ra khối 1 dài hơn k . Ngoài ra, nếu $\text{found} = \text{true}$ và $\text{len} = k - 1$, thì việc thêm một chữ 1 sẽ tạo ra thêm một khối dài đúng k thứ hai, nên nhánh này phải bị loại.

Điều kiện kết thúc

Khi đã xây xong toàn bộ xâu ($i = n + 1$), ta cần kiểm tra số khối 1 cực đại dài đúng k có trong xâu.

Có hai khả năng:

- một khối như vậy đã được khép lại trước đó, khi đó $\text{found} = \text{true}$;
- hoặc khối duy nhất dài đúng k nằm ở cuối xâu, khi đó $\text{len} = k$.

Vì vậy, ta đặt:

$$total = [found] + [len = k].$$

Xâu hợp lệ khi và chỉ khi

$$total = 1.$$

Giải mã

Algorithm 3 Backtracking liệt kê xâu có đúng một khối 1 cực đại dài k

```

1: function TRY( $i, len, found$ ) ▷  $i = 1..n + 1$ 
2:   if  $i = n + 1$  then
3:      $total \leftarrow 0$ 
4:     if  $found = \text{true}$  then
5:        $total \leftarrow total + 1$ 
6:     end if
7:     if  $len = k$  then
8:        $total \leftarrow total + 1$ 
9:     end if
10:    if  $total = 1$  then
11:      In xâu
12:    end if
13:    return
14:  end if ▷ Thử đặt 0

15:   $newFound \leftarrow found$ 
16:  if  $len = k$  then
17:     $newFound \leftarrow \text{true}$ 
18:  end if
19:   $x_i \leftarrow 0$ 
20:  TRY( $i + 1, 0, newFound$ ) ▷ Thử đặt 1

21:  if  $len + 1 \leq k$  then
22:    if not( $found = \text{true}$  and  $len = k - 1$ ) then
23:       $x_i \leftarrow 1$ 
24:      TRY( $i + 1, len + 1, found$ )
25:    end if
26:  end if
27: end function

```

Cách 2: Phương pháp sinh

Ý tưởng cấu trúc

Giải sử khối 1^k (gồm k số 1 liên tiếp) bắt đầu tại vị trí s :

$$1 \leq s \leq n - k + 1.$$

Để khối này là **cực đại** (không kéo dài được), ta phải có:

$$\text{nếu } s > 1 : x_{s-1} = 0, \quad \text{nếu } s + k - 1 < n : x_{s+k} = 0.$$

Khi đó xâu có dạng

$$L \text{ (nếu có 0) } 1^k \text{ (nếu có 0) } R.$$

Để đảm bảo **duy nhất** một dãy 1^k , cả L và R phải là các xâu **không chứa** mẫu 1^k (tức không có k số 1 liên tiếp).

Giả mã

Algorithm 4 Phương pháp sinh: cố định vị trí khối 1^k rồi sinh L, R

```

1:  $N \leftarrow n$ 
2: for  $s = 1$  to  $N - k + 1$  do
3:    $leftHas0 \leftarrow (s > 1)$ 
4:    $rightHas0 \leftarrow (s + k - 1 < N)$ 
5:    $leftTotal \leftarrow s - 1$ 
6:    $rightTotal \leftarrow N - (s + k - 1)$ 
7:    $\ell \leftarrow leftTotal - [leftHas0]$   $\triangleright [P] = 1$  nếu  $P$  đúng, ngược lại 0
8:    $r \leftarrow rightTotal - [rightHas0]$ 
9:   for mỗi xâu  $L$  độ dài  $\ell$  theo thứ tự từ điển do
10:    if  $L$  không chứa  $1^k$  then
11:      for mỗi xâu  $R$  độ dài  $r$  theo thứ tự từ điển do
12:        if  $R$  không chứa  $1^k$  then
13:          In  $L + (leftHas0?0 : \epsilon) + 1^k + (rightHas0?0 : \epsilon) + R$ 
14:        end if
15:      end for
16:    end if
17:  end for
18: end for

```

Cách sinh

Với mỗi vị trí có thể đặt khối 1^k :

- đặt khối 1^k vào đó;
- nếu bên trái còn chỗ thì đặt 0 ngay trước nó;
- nếu bên phải còn chỗ thì đặt 0 ngay sau nó;
- sinh mọi phần còn lại ở trái và phải;
- chỉ giữ những phần không chứa 1^k ;
- ghép lại và in ra.

5 Bài toán 2

Cho một hình chữ nhật gồm $n \times m$ hình vuông đơn vị. Hãy **liệt kê tất cả các đường đi** từ **đỉnh dưới cùng bên trái** đến **đỉnh trên cùng bên phải**. Biết rằng mỗi bước đi chỉ

được phép:

- dịch chuyển sang **bên phải** (kí hiệu R), hoặc
- dịch chuyển **lên trên** (kí hiệu U),

theo các cạnh của các hình vuông đơn vị.

Ví dụ

Với $n = 2, m = 3$, một số đường đi là:

$$RRRUU, \quad RRURU, \quad RURRU, \quad URRUR.$$

Mỗi xâu biểu diễn thứ tự các bước đi.

Nhận xét

Để đi từ góc dưới trái đến góc trên phải của hình chữ nhật $n \times m$, ta luôn phải:

- đi lên đúng n lần;
- đi sang phải đúng m lần.

Vì vậy, mỗi đường đi tương ứng với một xâu độ dài $n + m$ gồm đúng:

- n kí tự U ,
- m kí tự R .

Do đó, bài toán trở thành:

Liệt kê tất cả các xâu độ dài $n + m$ có đúng n kí tự U và đúng m kí tự R .

Cách 1: Quay lui (Backtracking)

Ý tưởng

Ta xây dựng đường đi từng bước một, từ trái sang phải trong xâu biểu diễn đường đi. Mỗi bước chỉ có thể là:

- R (đi sang phải),
- U (đi lên trên).

Để đi từ góc dưới trái đến góc trên phải của hình chữ nhật $n \times m$, ta cần:

- đúng m bước R ;
- đúng n bước U .

Vì vậy, tại mỗi bước:

- nếu chưa dùng hết m bước sang phải thì có thể thêm R ;
- nếu chưa dùng hết n bước đi lên thì có thể thêm U .

Giải thích trạng thái

Gọi

$$\text{Try}(i, r, u),$$

trong đó:

- i là vị trí đang chuẩn bị điền trong xâu đường đi;
- r là số bước sang phải R đã dùng;
- u là số bước đi lên U đã dùng.

Nói cách khác, ở thời điểm gọi $\text{Try}(i, r, u)$, ta đã xây dựng xong $i - 1$ bước đầu tiên của đường đi, trong đó có r chữ R và u chữ U .

Giải mã

Algorithm 5 Backtracking liệt kê các đường đi trong lưới $n \times m$

```

1: function TRY( $i, r, u$ )                                ▷  $i = 1..n + m + 1$ 
2:   if  $i = n + m + 1$  then
3:     In xâu hiện tại
4:     return
5:   end if
6:   if  $r < m$  then
7:      $x_i \leftarrow R$ 
8:     TRY( $i + 1, r + 1, u$ )
9:   end if
10:  if  $u < n$  then
11:     $x_i \leftarrow U$ 
12:    TRY( $i + 1, r, u + 1$ )
13:  end if
14: end function

```

Giải thích từng nhánh

- **Nhánh thêm R :** nếu $r < m$, nghĩa là ta vẫn còn quyền đi sang phải, nên có thể đặt $x_i = R$ và tiếp tục với trạng thái mới $(i + 1, r + 1, u)$.
- **Nhánh thêm U :** nếu $u < n$, nghĩa là ta vẫn còn quyền đi lên, nên có thể đặt $x_i = U$ và tiếp tục với trạng thái mới $(i + 1, r, u + 1)$.

Điều kiện kết thúc

Khi $i = n + m + 1$, tức là ta đã xây dựng đủ $n + m$ bước, thì xâu hiện tại chính là một đường đi hoàn chỉnh từ góc dưới trái đến góc trên phải, nên ta in ra.

Cách 2: Phương pháp sinh

Ý tưởng

Mỗi đường đi là một xâu độ dài $n + m$ gồm đúng n chữ U và m chữ R . Vì vậy, thay vì sinh từng bước đi, ta chỉ cần chọn xem các chữ U xuất hiện ở những vị trí nào.

Cụ thể, nếu chọn được một bộ chỉ số tăng dần

$$1 \leq p_1 < p_2 < \cdots < p_n \leq n + m,$$

thì ta tạo được đúng một đường đi bằng cách:

- đặt U tại các vị trí $p_1, p_2, \dots, p_n$;
- đặt R ở tất cả các vị trí còn lại.

Ngược lại, mỗi đường đi cũng xác định duy nhất một bộ vị trí xuất hiện của chữ U .

Do đó, bài toán trở thành:

Liệt kê tất cả các tổ hợp chập n của tập $\{1, 2, \dots, n + m\}$.

Sinh tổ hợp kế tiếp

Ta biểu diễn một tổ hợp bởi dãy tăng dần

$$(p_1, p_2, \dots, p_n).$$

Tổ hợp đầu tiên là

$$(1, 2, \dots, n).$$

Để sinh tổ hợp kế tiếp, ta làm như sau:

- tìm từ phải sang trái chỉ số lớn nhất i sao cho $p_i < m + i$;
- tăng p_i lên 1;
- đặt lại các phần tử phía sau theo giá trị nhỏ nhất có thể:

$$p_{i+1} = p_i + 1, \quad p_{i+2} = p_{i+1} + 1, \quad \dots, \quad p_n = p_{n-1} + 1.$$

Nếu không còn chỉ số nào thỏa $p_i < m + i$, thì ta đã đến tổ hợp cuối cùng và dừng.

Giải thích ngắn gọn

Trong thuật toán:

- dãy $(p_1, p_2, \dots, p_n)$ biểu diễn các vị trí của chữ U ;
- sau khi in xong tương ứng, ta chuyển sang tổ hợp kế tiếp;
- nhờ điều kiện tăng dần $p_1 < p_2 < \dots < p_n$, mỗi đường đi được sinh đúng một lần, không bị lặp.

Giải mã

Algorithm 6 Phương pháp sinh bằng tổ hợp vị trí của U

```
1: Khởi tạo  $p_1 \leftarrow 1, p_2 \leftarrow 2, \dots, p_n \leftarrow n$ 
2:  $done \leftarrow \text{false}$ 
3: while not  $done$  do
4:   Khởi tạo xâu  $x$  độ dài  $n + m$  gồm toàn kí tự  $R$ 
5:   for  $j = 1$  to  $n$  do
6:      $x_{p_j} \leftarrow U$ 
7:   end for
8:   In  $x$ 
9:    $i \leftarrow n$ 
10:  while  $i \geq 1$  and  $p_i = m + i$  do
11:     $i \leftarrow i - 1$ 
12:  end while
13:  if  $i = 0$  then
14:     $done \leftarrow \text{true}$ 
15:  else
16:     $p_i \leftarrow p_i + 1$ 
17:    for  $j = i + 1$  to  $n$  do
18:       $p_j \leftarrow p_{j-1} + 1$ 
19:    end for
20:  end if
21: end while
```
